CySA+ CS0-003
Portfolio Project Guide

8 hands-on projects | Step-by-step setup | GitHub deliverables | Exam objective mapping

Open Source Contributor
github.com/josecoss-dev

Prepared May 2026

Security ops 33%	Vuln mgmt 30%	Incident response 20%	Reporting 17%

Table of contents

Project 1

Wazuh SIEM home lab

[Security ops] [Incident response] Intermediate | 2-3 weeks | Impact: Very high

Deploy a full open-source SIEM/XDR stack used in production SOCs. By the end you'll have a Wazuh server ingesting logs from two endpoints, custom detection rules you wrote, documented attack simulations, and a GitHub repo that demonstrates real analyst skills.

System requirements

Component	Minimum	Recommended
Host RAM	12 GB	16 GB
Host storage	100 GB free	200 GB SSD
Hypervisor	VirtualBox (free)	VMware Workstation
Internet	Required	Required

VM	OS	vCPU	RAM	Role
wazuh-server	Ubuntu Server 22.04 LTS	4	6 GB	Wazuh Manager + Indexer + Dashboard
agent-linux	Ubuntu Desktop 22.04	2	2 GB	Wazuh agent (Linux endpoint)
agent-windows	Windows 10/11 or Server 2022 Eval	2	4 GB	Wazuh agent (Windows)
attacker	Kali Linux (optional)	2	2 GB	Attack simulation source

Set all VMs to the same Host-Only adapter in VirtualBox so they communicate with each other but stay off your real network.

Phase 1 — Infrastructure setup

1.1 Create the Wazuh server VM

```
Name: wazuh-server | Type: Linux | Version: Ubuntu 22.04 LTS (64-bit)
RAM: 6144 MB | Disk: 60 GB (VDI, dynamic) | Network: Host-Only (vboxnet0)
During Ubuntu install:
- Hostname: wazuh-server
- User: wazuh (strong password)
- Enable OpenSSH server: YES
- Note the IP after boot: ip a (example: 192.168.56.10)
```

1.2 Create agent VMs

Use the same Host-Only adapter for both. Windows: use the free 90-day Windows 10 Enterprise Evaluation or 180-day Server 2022 Evaluation ISO from Microsoft. Disable Defender real-time protection temporarily for the simulation phase.

Phase 2 — Wazuh server installation

2.1 System prep

```
sudo apt update && sudo apt upgrade -y
sudo apt install curl wget git -y
```

2.2 All-in-one install (installs Manager + Indexer + Dashboard)

```
curl -sO https://packages.wazuh.com/4.9/wazuh-install.sh
sudo bash ./wazuh-install.sh -a
```

This takes 10-15 minutes. When finished the installer prints admin credentials. Save them immediately. Access the dashboard at https://192.168.56.10 from your host browser.

2.3 Verify all services running

```
sudo systemctl status wazuh-manager
sudo systemctl status wazuh-indexer
sudo systemctl status wazuh-dashboard
```

Phase 3 — Agent deployment

3.1 Linux agent (run on agent-linux)

```
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | sudo gpg \
--no-default-keyring \
--keyring gnupg-ring:/usr/share/keyrings/wazuh.gpg \
--import && sudo chmod 644 /usr/share/keyrings/wazuh.gpg
echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] \
https://packages.wazuh.com/4.x/apt/ stable main" \
| sudo tee /etc/apt/sources.list.d/wazuh.list
sudo apt update
sudo WAZUH_MANAGER="192.168.56.10" apt install wazuh-agent -y
sudo systemctl daemon-reload && sudo systemctl enable --now wazuh-agent
```

3.2 Windows agent (PowerShell as Administrator)

```
Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.9.0-1.msi `
-OutFile wazuh-agent.msi
msiexec.exe /i wazuh-agent.msi /q `
WAZUH_MANAGER="192.168.56.10" `
WAZUH_AGENT_NAME="agent-windows"
NET START WazuhSvc
```

Verify in the Wazuh Dashboard under Agents — both should show Active. If Disconnected, open ports 1514 (TCP/UDP) and 1515 (TCP) on the server firewall.

Phase 4 — Log sources and custom detection rules

4.1 Additional log collection (on agent-linux, edit /var/ossec/etc/ossec.conf)

```
<localfile>
<log_format>syslog</log_format>
<location>/var/log/auth.log</location>
</localfile>
<localfile>
<log_format>syslog</log_format>
<location>/var/log/syslog</location>
</localfile>
```

4.2 File Integrity Monitoring (FIM) — add to ossec.conf

```
<syscheck>
<frequency>300</frequency>
<directories check_all="yes" realtime="yes">/etc,/usr/bin,/usr/sbin</directories>
<directories check_all="yes" realtime="yes">/home/wazuh/sensitive</directories>
</syscheck>
```

4.3 Custom rules (create /var/ossec/etc/rules/local_rules.xml on server)

```
<group name="local_rules,">
<!-- SSH brute force: 5 failures in 60 seconds -->
<rule id="100001" level="10" frequency="5" timeframe="60">
<if_matched_sid>5760</if_matched_sid>
<description>Possible SSH brute force attack detected</description>
<mitre><id>T1110.001</id></mitre>
</rule>
<!-- New local user account created -->
<rule id="100002" level="8">
<if_sid>5901</if_sid>
<description>New user account created on $(hostname)</description>
<mitre><id>T1136.001</id></mitre>
</rule>
<!-- FIM alert in sensitive directory -->
<rule id="100005" level="12">
<if_sid>550</if_sid>
<field name="file">/home/wazuh/sensitive</field>
<description>File modified in sensitive directory: $(file)</description>
<mitre><id>T1565.001</id></mitre>
</rule>
</group>
```

```
sudo systemctl restart wazuh-manager
```

Phase 5 — Attack simulations

Simulation	Command / Method	Expected alert	ATT&CK; TTP
SSH brute force	hydra -l wazuh -P rockyou.txt ssh://192.168.56.11	Rule 100001 level 10	T1110.001
FIM trigger	echo "x" >> ~/sensitive/creds.txt	Rule 100005 level 12	T1565.001
New user creation	sudo useradd testattacker	Rule 100002 level 8	T1136.001
PowerShell encoded	powershell -EncodedCommand	Event ID 4104 alert	T1059.001
Port scan	nmap -sS -A 192.168.56.11	Connection anomaly alerts	T1046

Phase 6 — Scripts to build and push to GitHub

scripts/alert_summary.py — queries Wazuh API and prints a daily summary

```
#!/usr/bin/env python3
import requests, argparse, urllib3
from collections import Counter
from datetime import datetime, timedelta
urllib3.disable_warnings()
def get_token(host, user, pw):
r = requests.post(f"https://{host}:55000/security/user/authenticate",
```

```
auth=(user, pw), verify=False)
return r.json()["data"]["token"]
def get_alerts(host, token, hours=24):
since = (datetime.utcnow()-timedelta(hours=hours)).strftime("%Y-%m-%dT%H:%M:%SZ")
hdrs = {"Authorization": f"Bearer {token}"}
r = requests.get(f"https://{host}:55000/alerts?limit=500&q=timestamp>{since}",
headers=hdrs, verify=False)
return r.json()["data"]["affected_items"]
def summarize(alerts):
rules = Counter(a.get("rule",{}).get("description","?") for a in alerts)
agents = Counter(a.get("agent",{}).get("name","?") for a in alerts)
print(f"\nWazuh Daily Summary | {len(alerts)} alerts")
print("Top rules:")
for d, c in rules.most_common(5): print(f" {c:>4}x {d}")
print("Most active agents:")
for a, c in agents.most_common(3): print(f" {c:>4} {a}")
if __name__ == "__main__":
ap = argparse.ArgumentParser()
ap.add_argument("--host", required=True)
ap.add_argument("--user", default="admin")
ap.add_argument("--password", required=True)
args = ap.parse_args()
token = get_token(args.host, args.user, args.password)
alerts = get_alerts(args.host, token)
summarize(alerts)
```

scripts/deploy_linux_agent.sh

```
#!/bin/bash
# Usage: ./deploy_linux_agent.sh <wazuh-server-ip>
WAZUH_IP=$1
[ -z "$WAZUH_IP" ] && echo "Usage: $0 <ip>" && exit 1
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | sudo gpg \
--no-default-keyring \
--keyring gnupg-ring:/usr/share/keyrings/wazuh.gpg --import
sudo chmod 644 /usr/share/keyrings/wazuh.gpg
echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] \
https://packages.wazuh.com/4.x/apt/ stable main" \
| sudo tee /etc/apt/sources.list.d/wazuh.list
sudo apt update
sudo WAZUH_MANAGER="$WAZUH_IP" apt install wazuh-agent -y
sudo systemctl enable --now wazuh-agent
echo "Done. Agent connecting to $WAZUH_IP"
```

GitHub repository structure

```
wazuh-siem-lab/
■■■ README.md <- Architecture overview + dashboard screenshots
■■■ setup/INSTALL.md <- Your step-by-step install notes
■■■ configs/
■ ■■■ ossec_linux_agent.conf <- Annotated agent config
■ ■■■ ossec_windows_agent.conf
■■■ rules/local_rules.xml <- Your custom detection rules (annotated)
■■■ scripts/
■ ■■■ alert_summary.py <- API summary tool
■ ■■■ deploy_linux_agent.sh
■ ■■■ deploy_windows_agent.ps1
■■■ simulations/
■ ■■■ brute_force_sim.md <- Command used + alert screenshot
```

```
    ■ ■■■ fim_sim.md
    ■ ■■■ user_creation_sim.md
    ■ ■■■ powershell_sim.md
    ■ ■■■ port_scan_sim.md
    ■■■ reports/
    ■■■ incident_YYYYMMDD_bruteforce.md
    ■■■ incident_YYYYMMDD_fim.md
    ■■■ ...one report per simulation
```

Write one incident report per simulation. Five simulations = five reports. This is the highest-value GitHub deliverable in this project — most candidates build the lab and skip the reports entirely. Don't be that person.

CySA+ CS0-003 objective mapping

Lab activity	CS0-003 objective	Domain
Dashboard alert analysis	1.2 Analyze indicators of malicious activity	Security ops
Custom detection rules	1.3 Use tools to determine malicious activity	Security ops
Log source configuration	1.1 System/network architecture in SecOps	Security ops
MITRE ATT&CK; in rules	1.4 Threat intelligence and threat hunting	Security ops
FIM + Vulnerability Detector	2.1 Implement vulnerability scanning	Vuln mgmt
Wazuh CVE module	2.3 Analyze data to prioritize vulnerabilities	Vuln mgmt
Five incident reports	3.2 Perform incident response activities	Incident response
ATT&CK; TTP mapping	3.1 Attack methodology frameworks	Incident response
Compliance dashboards	4.2 Compliance and assessment concepts	Reporting

Project 2

Vuln scan & remediation pipeline

[Vuln mgmt] Beginner | 1-2 weeks | Impact: High

Run Nessus Essentials (free, up to 16 IPs) against deliberately vulnerable VMs, prioritize findings by CVSS score, write professional remediation reports, and automate the triage process with a Python script. This project maps almost entirely to Domain 2.

Tools required

Tool	Version	Cost	Purpose
Nessus Essentials	Latest	Free (register email)	Vulnerability scanner
Metasploitable 2	N/A	Free (SourceForge)	Deliberately vulnerable Linux target
Metasploitable 3	N/A	Free (Vagrant + VirtualBox)	Windows vulnerable target
Python 3.x	3.10+	Free	Automate CSV parsing and report generation
VirtualBox	Latest	Free	Hypervisor for the target VMs

Phase 1 — Set up Nessus Essentials

1.1 Install Nessus

```
# On Ubuntu (wazuh-server works, or a separate VM)
# 1. Go to https://www.tenable.com/products/nessus/nessus-essentials
# Register with your email to get a free activation code
# 2. Download the .deb package, then:
sudo dpkg -i Nessus-<version>-ubuntu1404_amd64.deb
sudo systemctl start nessusd
sudo systemctl enable nessusd
# Access at: https://localhost:8834
# Complete setup wizard, enter your activation code when prompted
```

1.2 Get Metasploitable 2 (the easiest vulnerable target)

```
# Download from SourceForge:
# https://sourceforge.net/projects/metasploitable/
# Extract the .zip, open the .vmdk in VirtualBox as a new VM:
# Type: Linux | Version: Ubuntu (64-bit)
# Storage: Use existing .vmdk file
# Network: Host-Only (same adapter as your other VMs)
# Default login: msfadmin / msfadmin
# Note the IP: ifconfig (example: 192.168.56.20)
```

Phase 2 — Run your first vulnerability scan

2.1 Create and run a Basic Network Scan in Nessus

In the Nessus dashboard: New Scan -> Basic Network Scan. Set the target to 192.168.56.20 (your Metasploitable IP). Leave all plugin families enabled. Launch the scan. It takes 10-20 minutes.

2.2 Review results

When complete, you'll see findings sorted by severity. Metasploitable 2 will show dozens of Critical and High vulnerabilities including: vsftpd backdoor (CVE-2011-2523), UnrealIRCd backdoor, Samba vulnerabilities, Tomcat default credentials, and more. Export results: Report -> Export -> CSV. Save as nessus_scan_YYYYMMDD.csv.

Phase 3 — Python triage script

scripts/parse_nessus.py

```
#!/usr/bin/env python3
"""
parse_nessus.py -- parse Nessus CSV export, prioritize by CVSS,
output a markdown remediation report.
Usage: python3 parse_nessus.py nessus_scan_20260501.csv
"""
import csv, sys, datetime
from collections import defaultdict
SEVERITY_ORDER = {"Critical": 0, "High": 1, "Medium": 2, "Low": 3, "None": 4}
SLA = {"Critical": "24 hours", "High": "7 days", "Medium": "30 days", "Low": "90 days"}
def load_csv(path):
findings = []
with open(path, newline="", encoding="utf-8-sig") as f:
for row in csv.DictReader(f):
if row.get("Risk","None") == "None": continue
findings.append(row)
return sorted(findings, key=lambda r: SEVERITY_ORDER.get(r.get("Risk","None"),4))
def write_report(findings, out_path):
counts = defaultdict(int)
for f in findings: counts[f["Risk"]] += 1
date = datetime.date.today().isoformat()
with open(out_path, "w") as out:
out.write(f"# Vulnerability Assessment Report\n")
out.write(f"**Date:** {date} \n**Analyst:** [Your Name] \n\n")
out.write("## Executive Summary\n")
for sev in ["Critical","High","Medium","Low"]:
out.write(f"- **{sev}:** {counts.get(sev,0)} findings\n")
out.write("\n## Findings (prioritized by CVSS)\n\n")
for f in findings:
sev = f.get("Risk","?")
name = f.get("Name","?")
host = f.get("Host","?")
cvss = f.get("CVSS v3.0 Base Score", f.get("CVSS","?"))
desc = f.get("Description","")[:200].replace("\n"," ")
sol = f.get("Solution","")[:200].replace("\n"," ")
sla = SLA.get(sev,"?")
out.write(f"### [{sev}] {name}\n")
out.write(f"- **Host:** {host} \n- **CVSS:** {cvss} \n")
out.write(f"- **Remediation SLA:** {sla}\n")
out.write(f"- **Description:** {desc}\n")
out.write(f"- **Solution:** {sol}\n\n")
print(f"Report written to {out_path}")
if __name__ == "__main__":
if len(sys.argv) < 2:
print("Usage: parse_nessus.py <nessus.csv>")
```

```
    sys.exit(1)
    findings = load_csv(sys.argv[1])
    out = sys.argv[1].replace(".csv", "_report.md")
    write_report(findings, out)
```

GitHub repository structure

```
    vuln-scan-lab/
    ■■■ README.md <- What you scanned, findings summary, screenshots
    ■■■ setup/INSTALL.md <- Nessus + Metasploitable setup notes
    ■■■ scripts/parse_nessus.py <- Your triage and report generator
    ■■■ scans/ <- Export raw Nessus .nessus files here
    ■■■ reports/
    ■■■ vuln_report_YYYYMMDD.md <- Generated report (one per scan)
    ■■■ remediation_tracker.md <- Track which findings you've "remediated"
```

CySA+ CS0-003 objective mapping

Lab activity	CS0-003 objective	Domain
Running Nessus scans	2.1 Implement vulnerability scanning methods	Vuln mgmt
Reviewing plugin output	2.2 Analyze output from vulnerability assessment tools	Vuln mgmt
CVSS prioritization in script	2.3 Analyze data to prioritize vulnerabilities	Vuln mgmt
Remediation SLA in report	2.4 Recommend controls to mitigate vulnerabilities	Vuln mgmt
Written remediation report	4.1 Vulnerability management reporting	Reporting

Project 3

Phishing triage CLI tool

[Security ops] [Incident response] Beginner | 1 week | Impact: High

Build a Python CLI that parses suspicious emails, extracts indicators of compromise (IOCs), queries the VirusTotal API, and outputs a structured triage report. This is the most immediately deployable project — it's useful at HUIT the day you finish it.

Tools and setup

1.1 Get a free VirusTotal API key

```
# Register at: https://www.virustotal.com/gui/join-us
# Free tier: 4 API requests per minute, 500 per day
# After login: click your avatar -> API Key -> copy it
```

1.2 Python environment

```
mkdir phish-triage && cd phish-triage
python3 -m venv .venv
source .venv/bin/activate # Linux/Mac
# .venv\Scripts\activate # Windows
pip install requests python-dotenv colorama
# Create .env file:
echo "VT_API_KEY=your_key_here" > .env
echo ".env" >> .gitignore # NEVER push your API key
```

Phase 2 — Build the tool

phish_triage.py — main entrypoint

```
#!/usr/bin/env python3
"""
phish_triage.py -- Phishing email triage tool
Usage: python3 phish_triage.py email.eml
python3 phish_triage.py --text "paste raw headers here"
"""
import sys, json, argparse
from utils.ioc_extractor import extract_iocs
from utils.vt_client import check_ioc
from utils.report import build_report
def load_email(path):
with open(path, "r", errors="replace") as f: return f.read()
def triage(raw_text):
print("[*] Extracting IOCs...")
iocs = extract_iocs(raw_text)
print(f" Found: {len(iocs['ips'])} IPs, {len(iocs['domains'])} domains, "
f"{len(iocs['urls'])} URLs, {len(iocs['hashes'])} hashes")
print("[*] Querying VirusTotal (rate-limited)...")
results = {}
for ip in iocs["ips"]:
results[ip] = check_ioc(ip, "ip")
for domain in iocs["domains"]:
results[domain] = check_ioc(domain, "domain")
```

```
    for url in iocs["urls"]:
    results[url] = check_ioc(url, "url")
    print("[*] Building report...")
    return build_report(iocs, results)
    if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("email_file", nargs="?", help="Path to .eml file")
    ap.add_argument("--text", help="Raw header/body text")
    ap.add_argument("--json", action="store_true", help="Output JSON")
    args = ap.parse_args()
    raw = load_email(args.email_file) if args.email_file else args.text
    if not raw: ap.print_help(); sys.exit(1)
    report = triage(raw)
    if args.json: print(json.dumps(report, indent=2))
    else:
    print("\n" + "="*60)
    print(f"VERDICT: {report['verdict']}")
    print(f"Malicious indicators: {report['malicious_count']}")
    print("="*60)
```

utils/ioc_extractor.py

```
    import re, email
    IP_RE = re.compile(r'\b(?:[0-9]{1,3}\.){3}[0-9]{1,3}\b')
    DOMAIN_RE = re.compile(r'\b(?:[a-zA-Z0-9-]+\.)+[a-zA-Z]{2,}\b')
    URL_RE = re.compile(r'https?://[^\s<>"{}|\\^`\[\]]+')
    HASH_RE = re.compile(r'\b[0-9a-fA-F]{32,64}\b')
    PRIVATE = re.compile(
    r'^(10\.|172\.(1[6-9]|2[0-9]|3[01])\.|192\.168\.|127\.)'
    )
    def extract_iocs(raw_text):
    try:
    msg = email.message_from_string(raw_text)
    headers = str(dict(msg.items()))
    body = ""
    if msg.is_multipart():
    for part in msg.walk():
    if part.get_content_type() == "text/plain":
    body += part.get_payload(decode=True).decode("utf-8","replace")
    else:
    body = msg.get_payload(decode=True).decode("utf-8","replace")
    text = headers + " " + body
    except Exception:
    text = raw_text
    ips = [i for i in set(IP_RE.findall(text)) if not PRIVATE.match(i)]
    domains = list(set(DOMAIN_RE.findall(text)) - set(ips))
    urls = list(set(URL_RE.findall(text)))
    hashes = list(set(HASH_RE.findall(text)))
    return {"ips": ips, "domains": domains, "urls": urls, "hashes": hashes}
```

utils/vt_client.py

```
    import requests, time, os
    from dotenv import load_dotenv
    load_dotenv()
    VT_KEY = os.getenv("VT_API_KEY","")
    VT_BASE = "https://www.virustotal.com/api/v3"
    HEADERS = {"x-apikey": VT_KEY}
    def check_ioc(value, ioc_type):
    """Query VirusTotal for an IP, domain, or URL. Returns dict with verdict."""
```

```
endpoints = {
"ip": f"{VT_BASE}/ip_addresses/{value}",
"domain": f"{VT_BASE}/domains/{value}",
"url": f"{VT_BASE}/urls/{requests.utils.quote(value, safe='')}",
}
try:
r = requests.get(endpoints[ioc_type], headers=HEADERS, timeout=10)
time.sleep(15) # free tier: 4 req/min
if r.status_code == 200:
stats = r.json()["data"]["attributes"]["last_analysis_stats"]
malicious = stats.get("malicious", 0)
suspicious = stats.get("suspicious", 0)
return {"malicious": malicious, "suspicious": suspicious,
"verdict": "MALICIOUS" if malicious > 2 else
"SUSPICIOUS" if malicious + suspicious > 0 else "CLEAN"}
except Exception as e:
return {"error": str(e), "verdict": "ERROR"}
return {"verdict": "UNKNOWN"}
```

utils/report.py

```
from datetime import datetime
def build_report(iocs, vt_results):
malicious = sum(1 for v in vt_results.values() if v.get("verdict")=="MALICIOUS")
suspicious = sum(1 for v in vt_results.values() if v.get("verdict")=="SUSPICIOUS")
verdict = "MALICIOUS" if malicious > 0 else \
"SUSPICIOUS" if suspicious > 0 else "CLEAN"
return {
"timestamp": datetime.utcnow().isoformat(),
"verdict": verdict,
"malicious_count": malicious,
"suspicious_count": suspicious,
"iocs": iocs,
"vt_results": vt_results
}
```

Where to get test phishing emails

- PhishTank.com — downloadable phishing URL database
- OpenPhish.com — live phishing feed
- TryHackMe "Phishing Emails" rooms include safe sample .eml files
- MXToolbox Email Header Analyzer — paste headers to validate your parser

GitHub repository structure

```
phish-triage/
■■■ README.md <- Demo output screenshot + usage examples
■■■ phish_triage.py <- Main CLI entrypoint
■■■ utils/
■ ■■■ ioc_extractor.py <- Regex IOC extraction
■ ■■■ vt_client.py <- VirusTotal API wrapper
■ ■■■ report.py <- Report builder
■■■ tests/
■ ■■■ test_extractor.py <- Unit tests for IOC extractor
■ ■■■ samples/ <- Safe phishing email samples (.eml)
■■■ reports/sample_triage_report.md <- Example output for README
■■■ requirements.txt <- requests, python-dotenv, colorama
■■■ .gitignore <- Includes .env !
```

CySA+ CS0-003 objective mapping

Lab activity	CS0-003 objective	Domain
Email header parsing	1.2 Analyze indicators of malicious activity	Security ops
IOC extraction (IPs, domains, URLs)	1.3 Use tools to determine malicious activity	Security ops
VirusTotal API queries	1.4 Threat intelligence concepts	Security ops
Triage report output	3.2 Perform incident response activities	Incident response
ATT&CK; phishing mapping	3.1 Attack methodology frameworks	Incident response

Project 4

Threat hunt: BOTS v3 dataset

[Security ops] Intermediate | 2-3 weeks | Impact: Very high

Boss of the SOC (BOTS) is a dataset of real attack traffic created by the Splunk Security Research Team for their annual competition. You load it into Splunk and hunt adversary activity the same way a SOC analyst would. This is the highest-signal portfolio project because it shows employers you can work with real data and write professional threat hunt reports.

Phase 1 — Install Splunk and load the dataset

1.1 Download and install Splunk Enterprise (free)

```
# Download Splunk Enterprise from:
# https://www.splunk.com/en_us/download/splunk-enterprise.html
# (free license: 500 MB/day ingestion, no expiry on locally-loaded data)
# On Ubuntu:
wget -O splunk.deb "https://download.splunk.com/products/splunk/releases/
9.x.x/linux/splunk-9.x.x-linux-2.6-amd64.deb"
sudo dpkg -i splunk.deb
sudo /opt/splunk/bin/splunk start --accept-license
# Access at: http://localhost:8000
# Default user: admin (you set password on first login)
```

1.2 Download the BOTS v3 dataset

```
git clone https://github.com/splunk/botsv3.git
cd botsv3
# Follow the README to load the index into Splunk
# The dataset covers a simulated APT attack chain:
# Reconnaissance -> Weaponization -> Delivery -> Exploitation
# -> Installation -> C2 -> Actions on Objectives
```

1.3 Verify data loaded

```
index=botsv3 | stats count by sourcetype | sort -count
```

You should see sourcetypes including: WinEventLog:Security, WinEventLog:Microsoft-Windows-Sysmon/Operational, stream:http, stream:dns, XmlWinEventLog, suricata, and others. Total events: ~15 million.

Phase 2 — Hunt exercises with SPL queries

Hunt 1 — Detect reconnaissance / port scanning

```
index=botsv3 sourcetype=stream:tcp
| stats dc(dest_port) as ports_scanned count by src_ip
| where ports_scanned > 20
| sort -ports_scanned
```

What you're looking for: a single source IP connecting to many destination ports in a short window. This is the signature of an Nmap-style port scan. ATT&CK;: T1046 Network Service Discovery.

Hunt 2 — Find encoded PowerShell execution

```
index=botsv3 sourcetype="WinEventLog:Microsoft-Windows-PowerShell/Operational"
```

```
| search "*-EncodedCommand*" OR "*-enc *" OR "*-e *"
| table _time, host, user, ScriptBlockText
| sort _time
```

Attackers encode PowerShell commands in Base64 to evade simple string-matching. EventCode 4104 (Script Block Logging) records the decoded text. ATT&CK;: T1059.001.

Hunt 3 — Detect successful lateral movement (pass-the-hash / network logons)

```
index=botsv3 sourcetype="WinEventLog:Security" EventCode=4624
| where LogonType=3 AND NOT like(TargetUserName, "%$")
| stats count by src_ip, TargetUserName, ComputerName
| sort -count
```

LogonType 3 = network logon (lateral movement signature). Computer accounts (ending in $) are filtered as normal. ATT&CK;: T1021.002 (SMB/Windows Admin Shares).

Hunt 4 — Identify C2 beaconing by regularity of outbound connections

```
index=botsv3 sourcetype=stream:http
| bucket _time span=5m
| stats count by _time, src_ip, dest
| eventstats avg(count) as avg_count stdev(count) as stdev_count by src_ip, dest
| where stdev_count < 1 AND avg_count > 3
| sort src_ip, dest
```

C2 beaconing has very low standard deviation (consistent intervals). Low stdev + regular count = automated callback. ATT&CK;: T1071.001 (Web Protocols for C2).

Hunt 5 — Identify large outbound data transfers (exfiltration)

```
index=botsv3 sourcetype=stream:http method=POST
| stats sum(bytes_out) as total_bytes by src_ip, dest
| eval mb = round(total_bytes/1024/1024, 2)
| where mb > 1
| sort -mb
```

Large POST requests to external destinations may indicate data exfiltration. ATT&CK;: T1041 (Exfiltration Over C2 Channel).

Hunt 6 — Detect Mimikatz / credential dumping

```
index=botsv3 sourcetype="WinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=10
| search TargetImage="*lsass.exe"
| stats count by _time, SourceImage, SourceUser, ComputerName
| sort _time
```

Sysmon EventCode 10 = process access. lsass.exe holds credential hashes. Mimikatz and similar tools access lsass to dump credentials. ATT&CK;: T1003.001.

Phase 3 — Write hunt reports

For each hunt, write a report in Markdown. This is the deliverable employers want to see. Use this template:

```
# Threat Hunt Report: [Hunt Name]
**Date:** YYYY-MM-DD | **Analyst:** Your Name | **Dataset:** BOTS v3
## Hypothesis
We suspect [adversary activity] based on [initial indicator].
## Hunt methodology
1. Started with sourcetype X searching for Y
2. Narrowed by filtering Z
3. Pivoted on [IP/user/host] to confirm
## Findings
```

```
[What you found, with SPL query that proved it]
## ATT&CK mapping
- Tactic: [tactic]
- Technique: T[XXXX] [Technique name]
## IOCs identified
- IP: x.x.x.x
- Domain: evil.example.com
- Hash: [if applicable]
## Recommendations
[What detection rule or control would have caught this sooner]
```

GitHub repository structure

```
bots-threat-hunt/
■■■ README.md <- What BOTS is, what you hunted, key findings
■■■ splunk_queries/
■ ■■■ all_hunt_queries.spl <- Every SPL query with comments
■■■ hunt_reports/
■ ■■■ hunt_01_port_scan.md
■ ■■■ hunt_02_encoded_powershell.md
■ ■■■ hunt_03_lateral_movement.md
■ ■■■ hunt_04_c2_beaconing.md
■ ■■■ hunt_05_exfiltration.md
■ ■■■ hunt_06_credential_dump.md
■■■ methodology/
■■■ threat_hunt_process.md <- Your documented hunting methodology
```

CySA+ CS0-003 objective mapping

Lab activity	CS0-003 objective	Domain
Port scan detection query	1.2 Analyze indicators of malicious activity	Security ops
C2 beaconing hunt	1.3 Use tools to determine malicious activity	Security ops
ATT&CK; mapping in reports	1.4 Threat intelligence and threat hunting	Security ops
Hypothesis-led hunting	1.4 Threat hunting concepts	Security ops
Credential dump detection	1.2 Analyze indicators of malicious activity	Security ops
Written hunt reports	4.1 Reporting security findings	Reporting

Project 5

Malware traffic analysis (pcap)

[Security ops] [Vuln mgmt] Beginner | Ongoing — 1 per week | Impact: High

Analyze real malware packet captures, extract network IOCs, map activity to MITRE ATT&CK;, and write short analyst reports. This is the best weekly habit you can build — one pcap per week for 8 weeks gives you 8 portfolio write-ups and deep Wireshark fluency.

Phase 1 — Setup

1.1 Install tools

```
# Ubuntu
sudo apt update
sudo apt install wireshark tshark zeek -y
sudo usermod -aG wireshark $USER # allow non-root packet capture
# Python packages for automated IOC extraction
pip install pyshark scapy python-whois --break-system-packages
```

1.2 Where to get practice pcap files

Source	URL	Details
malware-traffic-analysis.net	malware-traffic-analysis.net	Best source. Exercises with answer keys. Password: infected
Wireshark sample captures	wiki.wireshark.org/SampleCaptures	Clean protocol examples
CyberDefenders.org	cyberdefenders.org	Blue team CTF challenges with pcap files
PacketTotal.com	packettotal.com	Upload pcaps, get analysis back
VirusTotal	virustotal.com	Upload pcap from a sandbox run

```
# Extract malware-traffic-analysis.net pcaps (always password: infected)
unzip -P infected 2024-xx-xx-traffic-analysis-exercise.zip
ls *.pcap
```

Phase 2 — Analysis workflow

2.1 Key Wireshark display filters

Filter	What it shows
http.request	All HTTP GET/POST requests — look for C2 callbacks, file downloads
dns	All DNS queries — look for DGA domains, C2 domain lookups
ssl.handshake.type == 1	TLS Client Hello — shows Server Name Indication (SNI) field
tcp.flags == 0x002	All SYN packets — shows all connection attempts
frame contains "MZ"	Packets containing PE file magic bytes (EXE download)

http.response.code == 200	Successful HTTP responses — what was delivered
ip.addr == x.x.x.x	Filter to/from a specific IP after identifying suspect host

2.2 Zeek analysis (automated protocol parsing)

```
# Point Zeek at a pcap — it creates structured logs automatically
mkdir zeek_output && cd zeek_output
zeek -r ../malware.pcap
# Logs created:
# conn.log - all connections (src/dst/bytes/duration)
# http.log - HTTP requests, user-agents, URIs, responses
# dns.log - DNS queries and answers
# ssl.log - TLS connections, SNI, cert subjects
# files.log - files transferred (with MD5/SHA1 hashes)
# weird.log - protocol anomalies
# Quick IOC extraction from Zeek logs:
cat dns.log | zeek-cut query | sort | uniq -c | sort -rn | head -20
cat http.log | zeek-cut host uri | sort | uniq
cat files.log | zeek-cut md5 sha1 mime_type | sort | uniq
```

2.3 Python IOC extractor from Zeek logs

```
#!/usr/bin/env python3
# pcap_ioc_extractor.py -- extract IOCs from Zeek logs
import os, csv
def read_zeek_log(path):
"""Read a Zeek TSV log file, skipping comment lines."""
rows = []
fields = []
with open(path) as f:
for line in f:
if line.startswith("#fields"):
fields = line.strip().split("\t")[1:]
elif not line.startswith("#"):
rows.append(dict(zip(fields, line.strip().split("\t"))))
return rows
def extract_iocs(zeek_dir):
iocs = {"domains":set(),"ips":set(),"urls":set(),"hashes":set(),"user_agents":set()}
# DNS domains
dns_log = os.path.join(zeek_dir, "dns.log")
if os.path.exists(dns_log):
for r in read_zeek_log(dns_log):
if q := r.get("query",""): iocs["domains"].add(q)
# HTTP URLs and user-agents
http_log = os.path.join(zeek_dir, "http.log")
if os.path.exists(http_log):
for r in read_zeek_log(http_log):
host = r.get("host",""); uri = r.get("uri","/")
if host: iocs["urls"].add(f"http://{host}{uri}")
if ua := r.get("user_agent",""): iocs["user_agents"].add(ua)
# File hashes
files_log = os.path.join(zeek_dir, "files.log")
if os.path.exists(files_log):
for r in read_zeek_log(files_log):
if h := r.get("md5",""): iocs["hashes"].add(h)
return {k: sorted(v) for k, v in iocs.items()}
if __name__ == "__main__":
import sys, json
```

```
result = extract_iocs(sys.argv[1] if len(sys.argv)>1 else ".")
print(json.dumps(result, indent=2))
```

Phase 3 — Write-up template (one per pcap)

```
# Malware Traffic Analysis: [Date] Exercise
**Analyst:** Your Name | **Date:** YYYY-MM-DD | **Source:** malware-traffic-analysis.net
## Summary
One paragraph: what happened, what malware family if identified, what was the impact.
## Network IOCs
| Type | Value | VT Hits | Notes |
|--------|----------------------|---------|--------------------|
| IP | 185.220.101.x | 45/90 | C2 server |
| Domain | malicious.example.com| 32/90 | DGA-generated |
| URL | http://evil.com/get | 28/90 | Payload delivery |
| Hash | a1b2c3... | 67/90 | Dropped EXE |
## Timeline
| Time (UTC) | Event |
|------------|---------------------------------|
| 14:32:01 | Initial HTTP GET to landing page |
| 14:32:15 | Redirect to exploit kit |
| 14:32:44 | EXE downloaded (MZ header in pcap)|
| 14:33:02 | C2 beacon begins (every 30s) |
## ATT&CK Mapping
- T1189 Drive-by Compromise (initial access)
- T1071.001 Web Protocols (C2 communication)
- T1041 Exfiltration Over C2 Channel
## Tools used
Wireshark, Zeek, pcap_ioc_extractor.py, VirusTotal
```

GitHub repository structure

```
malware-traffic-analysis/
■■■ README.md <- What this repo is, link to malware-traffic-analysis.net
■■■ scripts/
■ ■■■ pcap_ioc_extractor.py <- Your Zeek log parser
■■■ writeups/
■ ■■■ 2025-01-exercise.md <- One writeup per pcap exercise
■ ■■■ 2025-02-exercise.md
■ ■■■ ... <- Grow this every week
■■■ notes/
■■■ wireshark_cheatsheet.md <- Your personal Wireshark filter reference
```

CySA+ CS0-003 objective mapping

Lab activity	CS0-003 objective	Domain
Wireshark traffic analysis	1.2 Analyze indicators of malicious activity	Security ops
Zeek automated log parsing	1.3 Use tools to determine malicious activity	Security ops
C2 beaconing identification	1.2 Analyze indicators of malicious activity	Security ops
IOC extraction and VT lookup	1.4 Threat intelligence concepts	Security ops
ATT&CK; TTP mapping in write-ups	3.1 Attack methodology frameworks	Incident response

File hash analysis	2.2 Analyze vulnerability assessment tool output	Vuln mgmt

Project 6

Forensic investigation lab

[Incident response] Advanced | 3-4 weeks | Impact: Very high

Perform memory and disk forensics on compromised images, reconstruct attacker timelines, and deliver a formal forensic report. This is the project most candidates skip because it looks hard — which is exactly why it stands out in a portfolio review.

Tools required

Tool	Purpose	Install
Volatility 3	Memory forensics framework	git clone + pip install
Autopsy	Disk forensics GUI	Download from sleuthkit.org
strings / xxd	Binary analysis	sudo apt install binutils
TryHackMe	Practice memory images	Browser-based, free tier
CyberDefenders	Blue team CTF disk images	Browser-based, free tier

Phase 1 — Install Volatility 3

```
git clone https://github.com/volatilityfoundation/volatility3.git
cd volatility3
pip install -e . --break-system-packages
# Verify install:
python3 vol.py --help
# Download symbol tables for Windows memory analysis (required):
# https://github.com/volatilityfoundation/volatility3/releases
# Place .zip files in volatility3/volatility3/symbols/
```

Phase 2 — Where to get practice memory images

Source	URL	Notes
MemLabs	github.com/stuxnet999/MemLabs	6 free memory image challenges with write-ups
TryHackMe "Volatility"	tryhackme.com/room/volatility	Guided room, free tier, image provided
CyberDefenders "MemoryAnalysis"	cyberdefenders.org	Realistic SOC scenarios
CTFtime.org archives	ctftime.org	Search "forensics" memory challenges

Phase 3 — Memory forensics workflow (Volatility 3)

3.1 Essential commands — run these in order on every image

```
# Substitute memory.img with your actual file name
```

```
# 1. OS information (confirms profile, build, architecture)
python3 vol.py -f memory.img windows.info
# 2. Running processes (snapshot of what was running)
python3 vol.py -f memory.img windows.pslist
# 3. Process tree (reveals parent-child relationships)
python3 vol.py -f memory.img windows.pstree
# 4. Command line args (shows what commands were run)
python3 vol.py -f memory.img windows.cmdline
# 5. Network connections (C2, lateral movement)
python3 vol.py -f memory.img windows.netscan
# 6. Suspicious code injection (look for non-module-backed RWX memory)
python3 vol.py -f memory.img windows.malfind
# 7. Registry hives (persistence keys, run keys)
python3 vol.py -f memory.img windows.registry.hivelist
# 8. Dump a suspicious process for static analysis
python3 vol.py -f memory.img windows.dumpfiles --pid <PID>
# 9. Password hashes (for credential analysis)
python3 vol.py -f memory.img windows.hashdump
```

3.2 What to look for in each output

Command	Suspicious indicators	ATT&CK; TTP
pslist	Duplicate svchost.exe names, cmd.exe child of strange parent, explorer.exe spawned twice	T1055 Process Injection
cmdline	PowerShell -EncodedCommand, certutil -decode, wscript //nologo	T1059 Command Scripting
netscan	Unknown processes with ESTABLISHED connections to non-local IPs	T1071 C2 Channels
malfind	Regions marked MZ (PE header) in non-mapped memory	T1055.001 DLL Injection
hashdump	Password hashes extracted (compare against known user list)	T1003.001 LSASS Dump

3.3 Automation script: vol3_quick.sh

```
#!/bin/bash
# vol3_quick.sh -- run standard Volatility 3 triage against a memory image
# Usage: ./vol3_quick.sh memory.img output_dir
IMG=$1; OUTDIR=$2
[ -z "$IMG" ] && echo "Usage: $0 <image> <output_dir>" && exit 1
mkdir -p "$OUTDIR"
VOL="python3 /path/to/volatility3/vol.py -f $IMG"
echo "[*] Starting Volatility 3 triage: $IMG"
plugins=(
"windows.info"
"windows.pslist"
"windows.pstree"
"windows.cmdline"
"windows.netscan"
"windows.malfind"
"windows.registry.hivelist"
)
for plugin in "${plugins[@]}"; do
echo "[*] Running $plugin..."
outfile="$OUTDIR/${plugin//\./_}.txt"
```

```
$VOL $plugin > "$outfile" 2>&1
echo " Saved: $outfile"
done
echo "[+] Triage complete. Results in $OUTDIR"
```

Phase 4 — Disk forensics with Autopsy

4.1 Install Autopsy

Download from sleuthkit.org/autopsy. Free, GUI-based, cross-platform. For Linux, you can also use The Sleuth Kit (tsk) command-line tools: sudo apt install sleuthkit -y

4.2 Standard Autopsy workflow

```
1. Create new case: File -> New Case -> fill in case name/examiner
2. Add image: Add Data Source -> Disk Image or VM file
3. Configure ingest modules:
- Recent Activity (browser history, downloads, USB devices)
- Hash Lookup (flag known-bad files by hash)
- Keyword Search (search for email, passwords, attacker tools)
- File Type ID (find files with wrong extensions)
- Exif Parser (metadata from images)
4. Let ingest run (10-30 min depending on image size)
5. Review Results tree in left pane:
- Deleted Files
- Web History / Downloads
- Recent Documents
- Installed Programs
- Connected USB Devices
- Email Messages
6. Timeline view: Analyze -> Timeline (shows file activity over time)
7. Export key artifacts for your report
```

Phase 5 — Forensic report template

```
# Forensic Investigation Report
**Case #:** CASE-2025-001
**Examiner:** Your Name | **Date:** YYYY-MM-DD
**Evidence:** memory.img (MD5: xxxxxxxx) | disk.dd (MD5: xxxxxxxx)
## Executive Summary
2-3 sentences: what happened, when, impact.
## Scope and Methodology
What tools were used (Volatility 3.x, Autopsy x.x.x, tshark x.x).
What analysis was performed (memory analysis, disk analysis, timeline).
## Timeline of Events
| Time (UTC) | Event | Evidence |
|--------------|-----------------------------------------|--------------------|
| 2025-01-15 | Malicious email opened (LNK file) | Recent Docs |
| 2025-01-15 | PowerShell executes encoded payload | cmdline output |
| 2025-01-15 | C2 connection established to x.x.x.x | netscan output |
| 2025-01-16 | Lateral movement via SMB | Event log 4624 |
## Key Findings
### Finding 1: [Name]
Description, evidence source, ATT&CK mapping, screenshot reference.
## IOCs Identified
| Type | Value | Source |
|--------|-----------------|----------------|
| IP | 192.0.2.1 | netscan.txt |
| Hash | abc123... | files.log |
```

```
## Conclusions
## Appendices
A. Evidence hash values
B. Full Volatility output
C. Tool versions
```

GitHub repository structure

```
forensic-investigation-lab/
■■■ README.md <- What you analyzed, tools, key findings
■■■ scripts/
■ ■■■ vol3_quick.sh <- Volatility automation script
■■■ methodology/
■ ■■■ forensic_checklist.md <- Your step-by-step investigation checklist
■■■ reports/
■■■ forensic_report_template.md <- Blank template
■■■ case_YYYYMMDD/
■■■ forensic_report.md <- Your completed report
■■■ netscan.txt <- Volatility output files
■■■ pslist.txt
■■■ timeline.csv <- Autopsy timeline export
```

CySA+ CS0-003 objective mapping

Lab activity	CS0-003 objective	Domain
Memory acquisition/hashing	3.2 Perform incident response activities	Incident response
Volatility pslist/pstree	3.2 Analyze malware and indicators	Incident response
Timeline reconstruction	3.2 Perform incident response activities	Incident response
ATT&CK; TTP mapping	3.1 Attack methodology frameworks	Incident response
Formal forensic report	4.1 Reporting security findings	Reporting
Chain of custody documentation	3.3 Post-incident activity phases	Incident response

Project 7

TryHackMe SOC Level 1 path

[Security ops] [Vuln mgmt] [Incident response] [Reporting] Beginner | 4-6 weeks alongside course | Impact: Medium

TryHackMe provides guided, browser-based learning rooms with built-in VMs — zero lab setup. The SOC Level 1 learning path covers every CySA+ CS0-003 domain in practical, hands-on rooms. Run this in parallel with the Jason Dion Udemy course for maximum retention.

Free tier access: https://tryhackme.com | Subscription ($14/mo): unlocks all rooms. The free tier has enough content to complete most of the path. Start free, subscribe when you hit a paywall on a room you need.

SOC Level 1 path — room-by-room breakdown

Section 1: Cyber defence frameworks (Security ops / Reporting)

Room	What it covers	CySA+ domain	Free?
Jr SOC Analyst Intro	SOC roles, alert triage workflow, ticketing	Security ops	Yes
Pyramid of Pain	IOC types, attacker cost of each layer	Security ops	Yes
Cyber Kill Chain	Lockheed Martin 7-stage attack model	Security ops	Yes
Unified Kill Chain	Extended kill chain model	Security ops	Yes
Diamond Model	Diamond model for incident analysis	Security ops	Yes
MITRE	ATT&CK; framework, Navigator tool	Security ops	Yes

Section 2: Cyber threat intelligence

Room	What it covers	CySA+ domain	Free?
Intro to Cyber Threat Intel	TI concepts, tactical/strategic/operational	Security ops	Yes
Threat Intelligence Tools	UrlScan, Abuse.ch, PhishTool, Talos	Security ops	Yes
Yara	Write Yara rules to detect malware signatures	Security ops	Yes
OpenCTI	Threat intel platform hands-on	Security ops	Sub
MISP	MISP threat sharing platform	Security ops	Sub

Section 3: Network security and traffic analysis

Room	What it covers	CySA+ domain	Free?
Snort	IDS/IPS rules, alert tuning	Security ops	Yes
Snort Challenge	Apply Snort rules to real traffic	Security ops	Sub
NetworkMiner	PCAP analysis and artifact extraction	Security ops	Yes

Zeek	Zeek scripting and log analysis	Security ops	Sub
Wireshark: The Basics	Filters, follow streams, statistics	Security ops	Yes
Wireshark: Packet Ops	Advanced analysis, PBQs practice	Security ops	Sub

Section 4: Endpoint security monitoring (critical for CySA+)

Room	What it covers	CySA+ domain	Free?
Core Windows Processes	Normal vs. suspicious process behavior	Security ops	Yes
Sysinternals	Process Monitor, Autoruns, TCPView	Security ops	Yes
Windows Event Logs	Event IDs 4624/4625/4688/4698/4720...	Security ops	Yes
Sysmon	Configure Sysmon, SwiftOnSecurity config	Security ops	Yes
Osquery	SQL-based endpoint querying	Security ops	Yes
Wazuh (THM room)	Wazuh agent deployment, rules, alerts	Security ops	Sub

Section 5: SIEM

Room	What it covers	CySA+ domain	Free?
Intro to SIEM	SIEM concepts, log sources, use cases	Security ops	Yes
Investigating with ELK	Elastic SIEM alert investigation	Security ops	Yes
ItsyBitsy (ELK)	Practical ELK threat hunting exercise	Security ops	Sub
Splunk: Basics	SPL fundamentals, searching, stats	Security ops	Yes
Investigating with Splunk	Full incident investigation in Splunk	Security ops	Sub
Benign	Identify benign vs. malicious in Splunk	Security ops	Sub

Section 6: Phishing analysis

Room	What it covers	CySA+ domain	Free?
Phishing Analysis Fundamentals	Email header analysis, IOC extraction	IR	Yes
Phishing Emails 1	Analyze a suspicious email (practical)	IR	Yes
Phishing Emails 2	Harder phishing analysis exercise	IR	Yes
Phishing Emails 3-5	Progressive difficulty challenges	IR	Sub
Phishing Analysis Tools	PhishTool, URL analysis, sandboxing	IR	Sub

How to document for GitHub

Write a room write-up for every completed room. Do NOT publish answers/flags for subscription rooms — this violates TryHackMe's terms of service. For free rooms, write-ups are acceptable and encouraged.

```
# Room write-up template
# writeups/room-name.md
# [Room Name]
**Platform:** TryHackMe | **Difficulty:** [Easy/Medium/Hard]
**Completed:** YYYY-MM-DD
## What this room covers
2-3 sentences on the topic.
## Key concepts learned
- Concept 1 and why it matters
- Concept 2 with an example
- ...
## Tools used
- Tool 1: brief note on what I used it for
## CySA+ CS0-003 mapping
This room covers objective [X.X] in the [Domain] domain because...
## Reflection
One thing that clicked for me in this room that I'd missed before.
```

GitHub repository structure

```
tryhackme-soc-path/
■■■ README.md <- TryHackMe profile link + path completion screenshot
■■■ writeups/
■ ■■■ cyber-kill-chain.md
■ ■■■ mitre-attack.md
■ ■■■ windows-event-logs.md
■ ■■■ sysmon.md
■ ■■■ wireshark-basics.md
■ ■■■ ... (one per completed room)
■■■ notes/
■■■ key_event_ids.md <- Windows Event IDs cheat sheet you build
■■■ spl_cheatsheet.md <- SPL queries reference
■■■ attack_techniques.md <- Running ATT&CK TTP notes
```

Recommended study order

Week	Focus rooms	Why this order
1	Jr SOC Analyst, Kill Chain, MITRE	Frameworks first — everything else builds on these
2	Windows Event Logs, Core Windows Processes, Sysmon	Endpoint visibility is 50% of the exam
3	Wireshark Basics, Snort, NetworkMiner	Network analysis skills
4	Splunk Basics, Intro to SIEM, CTI Tools	SIEM and threat intel
5	Phishing rooms 1-3, Threat Intel Tools	IR and phishing analysis
6	Zeek, Osquery, Investigating with Splunk	Tie it all together with practical exercises

CySA+ CS0-003 objective mapping

Room category	CS0-003 objectives covered	Domain
Frameworks (Kill Chain, ATT&CK;, Diamond)	3.1 Attack methodology frameworks	Incident response

Windows Event Logs, Sysmon	1.1 System architecture in security ops	Security ops
Wireshark, Zeek, NetworkMiner	1.2 Analyze indicators of malicious activity	Security ops
Splunk, ELK (SIEM rooms)	1.3 Use tools to determine malicious activity	Security ops
Threat Intel (CTI, MISP, OpenCTI)	1.4 Threat intelligence concepts	Security ops
Phishing analysis rooms	3.2 Perform incident response activities	Incident response

Project 8

Security metrics dashboard

[Reporting] [Vuln mgmt] Intermediate | 2-3 weeks | Impact: High

Build a Python Dash web app that visualizes security operations metrics: vulnerability trends, patch compliance, and alert counts. Deploy it publicly on Render.com (free tier). This project proves you can translate security data into business-readable visuals — a skill most technical candidates lack.

Phase 1 — Setup

1.1 Create the project

```
mkdir security-dashboard && cd security-dashboard
python3 -m venv .venv && source .venv/bin/activate
pip install dash pandas plotly
# Create requirements.txt:
pip freeze > requirements.txt
```

Phase 2 — Build the dashboard

app.py — main dashboard application

```
#!/usr/bin/env python3
"""
Security Operations Dashboard
Run: python3 app.py then open http://localhost:8050
"""
import dash
from dash import dcc, html, Input, Output, dash_table
import plotly.graph_objects as go
import plotly.express as px
import pandas as pd
from data.mock_data import get_vuln_data, get_alert_data, get_compliance_data
app = dash.Dash(__name__, title="SecOps Dashboard")
# ■■ Layout
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
app.layout = html.Div([
html.Div([
html.H1("Security Operations Dashboard",
style={"color":"#0C447C","fontFamily":"sans-serif","margin":"0"}),
html.P("Last updated: 2026-05-10",
style={"color":"#888","fontFamily":"sans-serif","margin":"4px 0 0"})
], style={"padding":"20px 30px","borderBottom":"2px solid #0C447C"}),
# Metric cards row
html.Div([
metric_card("Total vulnerabilities", "247", "#D85A30"),
metric_card("Critical / High", "38", "#D85A30"),
metric_card("Patch compliance", "73%", "#BA7517"),
metric_card("Alerts (7d)", "142", "#1D9E75"),
metric_card("Mean time to detect", "4.2h","#534AB7"),
], style={"display":"flex","gap":"16px","padding":"20px 30px"}),
# Charts row
html.Div([
html.Div([dcc.Graph(id="vuln-trend")],
```

```
    style={"flex":"2","background":"white","borderRadius":"8px",
    "border":"1px solid #ddd","padding":"16px"}),
    html.Div([dcc.Graph(id="severity-pie")],
    style={"flex":"1","background":"white","borderRadius":"8px",
    "border":"1px solid #ddd","padding":"16px","marginLeft":"16px"}),
    ], style={"display":"flex","padding":"0 30px 20px"}),
    # Bottom row
    html.Div([
    html.Div([dcc.Graph(id="top-hosts")],
    style={"flex":"1","background":"white","borderRadius":"8px",
    "border":"1px solid #ddd","padding":"16px"}),
    html.Div([dcc.Graph(id="compliance-gauge")],
    style={"flex":"1","background":"white","borderRadius":"8px",
    "border":"1px solid #ddd","padding":"16px","marginLeft":"16px"}),
    ], style={"display":"flex","padding":"0 30px 20px"}),
    ], style={"backgroundColor":"#F5F5F3","minHeight":"100vh"})
    def metric_card(label, value, color):
    return html.Div([
    html.P(label, style={"margin":"0","fontSize":"12px","color":"#666"}),
    html.H2(value, style={"margin":"4px 0 0","color":color,"fontSize":"28px"})
    ], style={"background":"white","borderRadius":"8px","padding":"16px",
    "border":f"2px solid {color}","flex":"1","textAlign":"center"})
    # ■■ Callbacks
    ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    @app.callback(
    [Output("vuln-trend","figure"), Output("severity-pie","figure"),
    Output("top-hosts","figure"), Output("compliance-gauge","figure")],
    Input("vuln-trend","id")
    )
    def update_charts(_):
    df_vuln = get_vuln_data()
    df_alert = get_alert_data()
    df_comp = get_compliance_data()
    # Vuln trend stacked bar
    fig_trend = go.Figure()
    colors = {"Critical":"#D85A30","High":"#BA7517","Medium":"#534AB7","Low":"#1D9E75"}
    for sev in ["Critical","High","Medium","Low"]:
    fig_trend.add_trace(go.Bar(
    x=df_vuln["week"], y=df_vuln[sev],
    name=sev, marker_color=colors[sev]
    ))
    fig_trend.update_layout(barmode="stack", title="Vulnerability trend (8 weeks)",
    plot_bgcolor="white", paper_bgcolor="white")
    # Severity pie
    latest = df_vuln.iloc[-1]
    fig_pie = go.Figure(go.Pie(
    labels=["Critical","High","Medium","Low"],
    values=[latest["Critical"],latest["High"],latest["Medium"],latest["Low"]],
    marker_colors=list(colors.values())
    ))
    fig_pie.update_layout(title="Current severity distribution",
    paper_bgcolor="white")
    # Top vulnerable hosts
    top_hosts = df_alert.groupby("host")["count"].sum().nlargest(10).reset_index()
    fig_hosts = px.bar(top_hosts, x="count", y="host", orientation="h",
    title="Top 10 hosts by alert count",
    color_discrete_sequence=["#0C447C"])
    fig_hosts.update_layout(paper_bgcolor="white", plot_bgcolor="white")
    # Patch compliance gauge
```

```
pct = df_comp["patched"].sum() / df_comp["total"].sum() * 100
fig_gauge = go.Figure(go.Indicator(
mode="gauge+number", value=round(pct, 1),
title={"text":"Patch compliance %"},
gauge={"axis":{"range":[0,100]},
"bar":{"color":"#1D9E75"},
"steps":[{"range":[0,60],"color":"#FAECE7"},
{"range":[60,80],"color":"#FAEEDA"},
{"range":[80,100],"color":"#E1F5EE"}]}
))
fig_gauge.update_layout(paper_bgcolor="white")
return fig_trend, fig_pie, fig_hosts, fig_gauge
if __name__ == "__main__":
app.run(debug=True)
```

data/mock_data.py — sample data (replace with real Nessus CSV output)

```
import pandas as pd, numpy as np
def get_vuln_data():
weeks = [f"W{i}" for i in range(1,9)]
np.random.seed(42)
return pd.DataFrame({
"week": weeks,
"Critical": np.random.randint(3,12,8),
"High": np.random.randint(10,35,8),
"Medium": np.random.randint(50,120,8),
"Low": np.random.randint(80,180,8),
})
def get_alert_data():
hosts = [f"host-{i:02d}" for i in range(1,21)]
np.random.seed(7)
return pd.DataFrame({
"host": hosts,
"count": np.random.randint(1,80,20),
})
def get_compliance_data():
return pd.DataFrame({
"severity": ["Critical","High","Medium","Low"],
"total": [10, 26, 98, 113],
"patched": [8, 18, 65, 89],
})
```

Phase 3 — Deploy to Render.com (free, public URL)

```
# 1. Push your repo to GitHub
git init && git add . && git commit -m "initial dashboard"
git remote add origin https://github.com/yourusername/security-dashboard.git
git push -u origin main
# 2. Create a Procfile at repo root:
echo "web: python app.py" > Procfile
# 3. Update app.py to bind to PORT env var:
# Replace: app.run(debug=True)
# With:
import os
port = int(os.environ.get("PORT", 8050))
app.run(host="0.0.0.0", port=port)
# 4. Go to render.com -> New Web Service -> Connect GitHub repo
# Build command: pip install -r requirements.txt
# Start command: python app.py
```

```
# Free tier: your dashboard stays live at https://your-app.onrender.com
```

Phase 4 — Extend with real data (connect to Project 2)

Replace mock_data.py functions with real data from your Nessus CSV output (generated by parse_nessus.py from Project 2). Add a file upload component to Dash that accepts a Nessus CSV and dynamically renders the charts from real scan data. This turns two separate projects into one integrated security toolchain.

GitHub repository structure

```
security-dashboard/
■■■ README.md <- Live URL link + screenshot + what metrics are shown
■■■ app.py <- Main Dash application
■■■ data/
■ ■■■ mock_data.py <- Sample data generator
■ ■■■ nessus_loader.py <- Real Nessus CSV loader (from Project 2)
■■■ requirements.txt
■■■ Procfile <- For Render.com deployment
■■■ assets/
■■■ style.css <- Optional custom CSS
```

CySA+ CS0-003 objective mapping

Lab activity	CS0-003 objective	Domain
Vulnerability trend chart	4.1 Vulnerability management reporting	Reporting
Patch compliance gauge	2.4 Recommend controls to mitigate vulns	Vuln mgmt
Severity distribution	2.3 Analyze data to prioritize vulns	Vuln mgmt
Alert count by host	1.2 Analyze indicators of malicious activity	Security ops
MTTD/MTTR metrics	4.1 Communicate security metrics to stakeholders	Reporting
Public deployment	4.2 Compliance and assessment (demo capability)	Reporting

Recommended study timeline

Week	Projects active	Milestone
1	Project 7 (THM: Frameworks rooms) + Project 1 setup	VMs up, Wazuh installed, THM account created
2	Project 1 (agents + rules) + Project 7 (Endpoint rooms)	Both agents active, first custom rule firing
3	Project 1 (simulations + reports) + Project 2 setup	5 incident reports written, Nessus installed
4	Project 2 (scan + Python script) + Project 7 (SIEM rooms)	Nessus report generated, Splunk rooms started
5	Project 3 (phishing tool) + Project 5 week 1	Phishing CLI working + pushed to GitHub
6	Project 4 (BOTS setup + hunts 1-3) + Project 5 week 2	3 threat hunt reports written
7	Project 4 (hunts 4-6) + Project 5 week 3	6 hunt reports, 3 pcap write-ups
8	Project 6 (Volatility + Autopsy) + Project 8 start	Forensic report complete
9-10	Project 8 (dashboard) + GitHub polish	Live dashboard deployed, all READMEs written
11-12	Review Dion course + practice exams	Practice exam score >= 80%, ready to sit

Priority order for your September deadline

Your Security+ renewal is time-sensitive. That gives roughly 4 months. Complete Projects 7, 1, 2, and 3 first — those four together cover all four CySA+ domains and produce the most GitHub commits. Then sit the exam. Projects 4-8 can continue after passing.

Priority	Project	Why this order
1st	TryHackMe SOC Level 1 (Project 7)	Zero setup, starts immediately, builds all domain knowledge in parallel
2nd	Wazuh SIEM lab (Project 1)	Highest impact project. Covers 60% of exam objectives. Do this early.
3rd	Phishing triage tool (Project 3)	1 week, ships to GitHub fast, directly applicable at work
4th	Vuln scan pipeline (Project 2)	Domain 2 coverage. Connects to dashboard later.
5th	BOTS threat hunt (Project 4)	Very high signal with employers. Start after Splunk rooms on THM.
6th	Pcap analysis (Project 5)	Weekly habit. Run alongside everything else.
7th	Security dashboard (Project 8)	Strongest engineering/security combo piece. Build last.
8th	Forensic lab (Project 6)	Advanced. High value but not exam-critical. Do after passing.

GitHub portfolio strategy

All 8 repos should follow the same structure: a strong README with screenshots, a clear description of what you built and why, and links to your reports. Pin all 8 repos on your GitHub profile page. Each repo title should include the tool/technology name so it's searchable: "wazuh-siem-lab", "nessus-vuln-pipeline", "phish-triage-cli", etc.

One optional addition: create a GitHub Pages site (github.io) that serves as your security portfolio index — links to all 8 repos, a one-paragraph summary of each, and your TryHackMe profile. This is the URL you put on your resume.

CEU credit reminder

Every hour of documented training in these projects counts as 1 CEU toward your Security+ renewal (which requires 50 CEUs). Log each project in your CompTIA Certification account at ce.comptia.org as "Self-Paced Training." You'll need: course/training name, provider (self-directed), completion date, and hours. Keep a running log so you're not scrambling at renewal time.

Better path: pass the CySA+ exam. That automatically renews Security+, waives the renewal fee, adds CySA+ to your credentials, and resets the 3-year clock on both certifications simultaneously.

Resource	URL	Purpose
TryHackMe	tryhackme.com	Guided labs (Projects 1,5,6,7)
Jason Dion CySA+ (Udemy)	udemy.com — search "Dion CySA+ CS0-003"	Primary exam prep course
malware-traffic-analysis.net	malware-traffic-analysis.net	Pcap exercises (Project 5)
Wazuh documentation	documentation.wazuh.com	Project 1 reference
BOTS v3 dataset	github.com/splunk/botsv3	Project 4 dataset
MemLabs	github.com/stuxnet999/MemLabs	Project 6 memory images
CyberDefenders	cyberdefenders.org	Projects 5 and 6 challenges
MITRE ATT&CK; Navigator	mitre-attack.github.io/attack-navigator	TTP mapping for all projects
CompTIA CE Portal	ce.comptia.org	Log CEUs, track renewal status

Built for CySA+ Community Guide University | CySA+ CS0-003 | May 2026